# CS 103000
# Prof. Madeline Blount

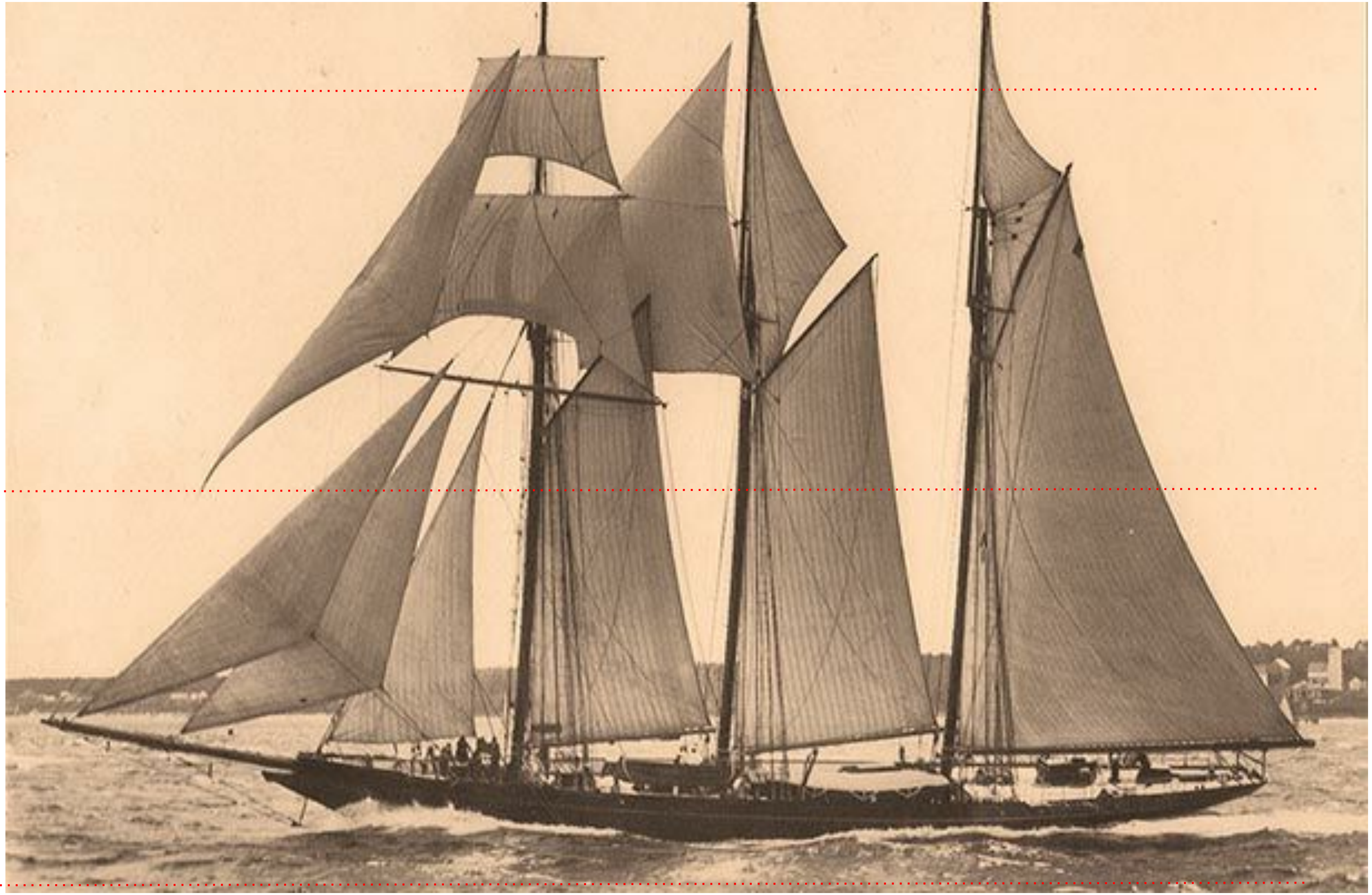
# Week 7:
# VECTORS (cont.)

*Dall-E 2: cats learning C++ in the forest on ‘90’s technology* https://shorturl.at/TET5C

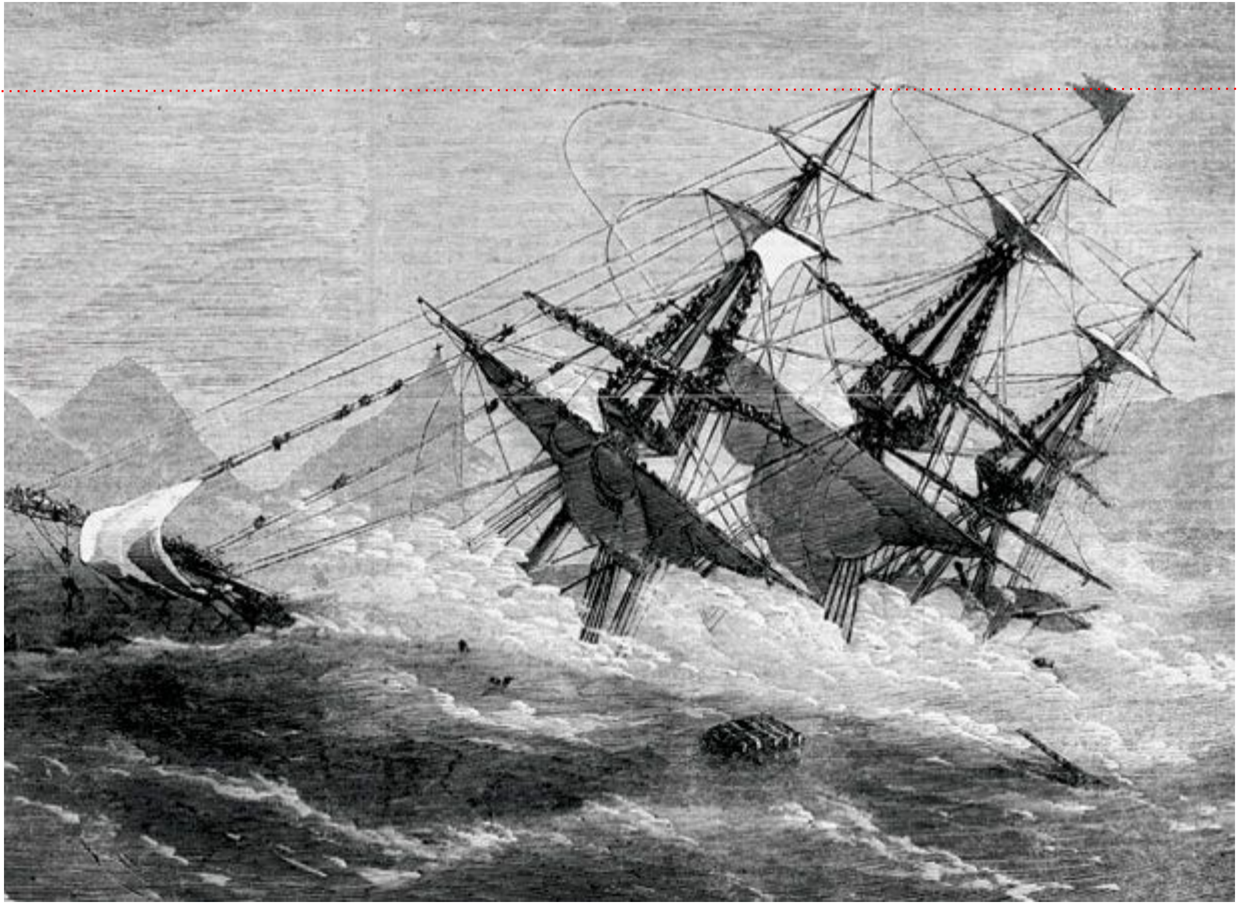
high-level

mid-level

low-level

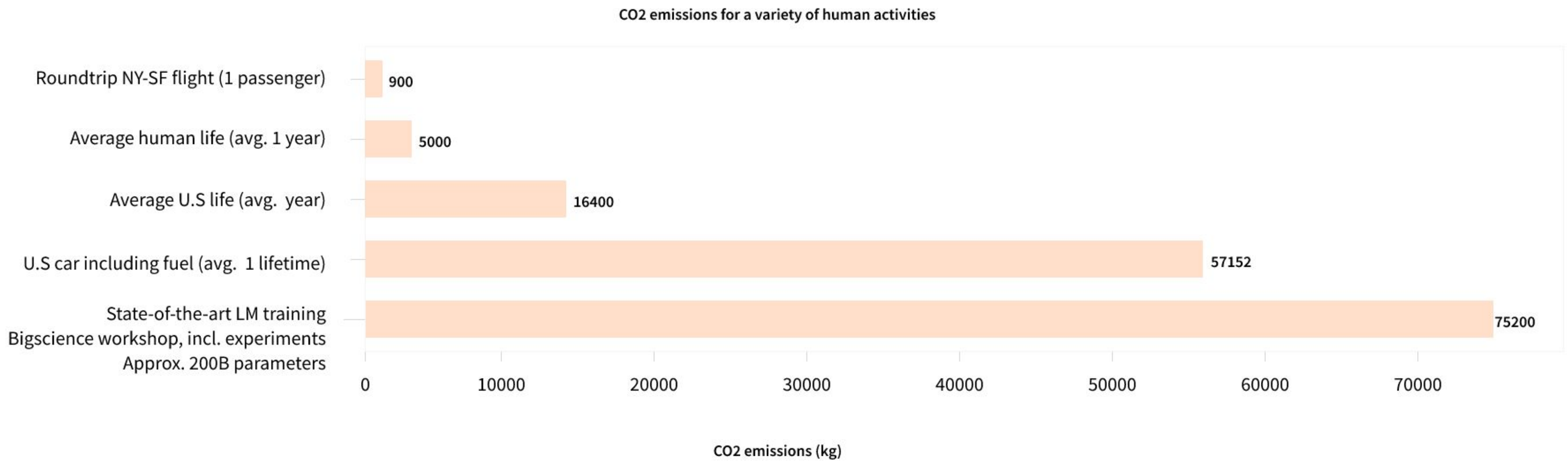


high-level??



## 🤖 MIDTERM!

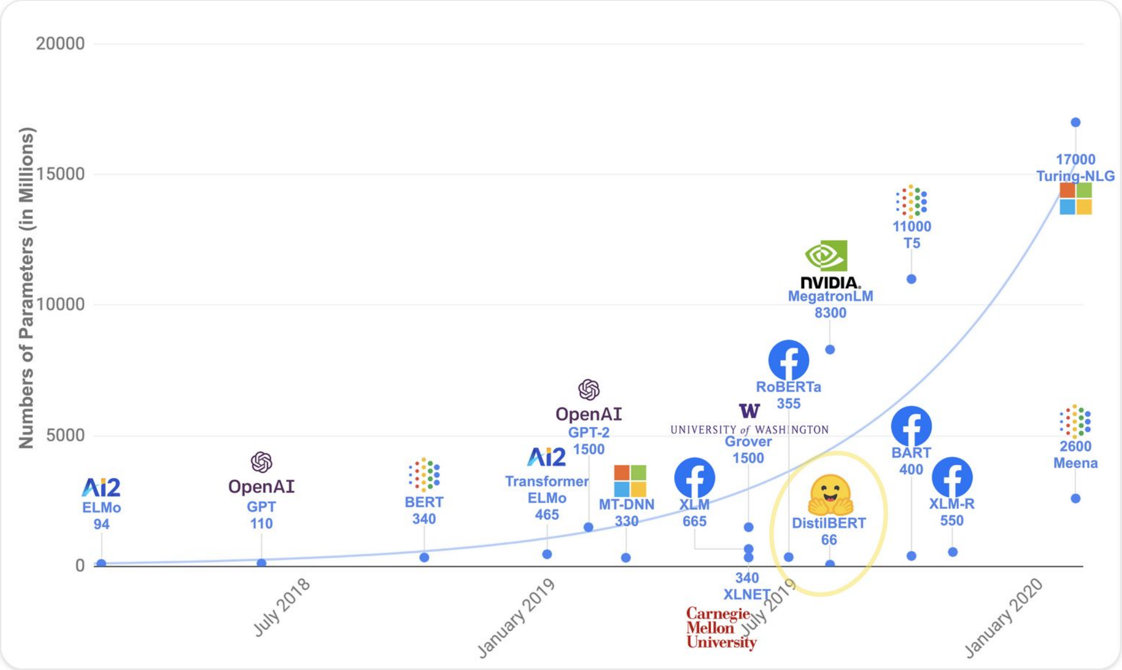
- **Review on 03.23, based on survey**
- **HACKATHON TEAMS also on 03.23, *attendance required***
- **In-class work 03.23, 03.25, Fri. 03.27**
- **Code due 03.27**




source: huggingface.co

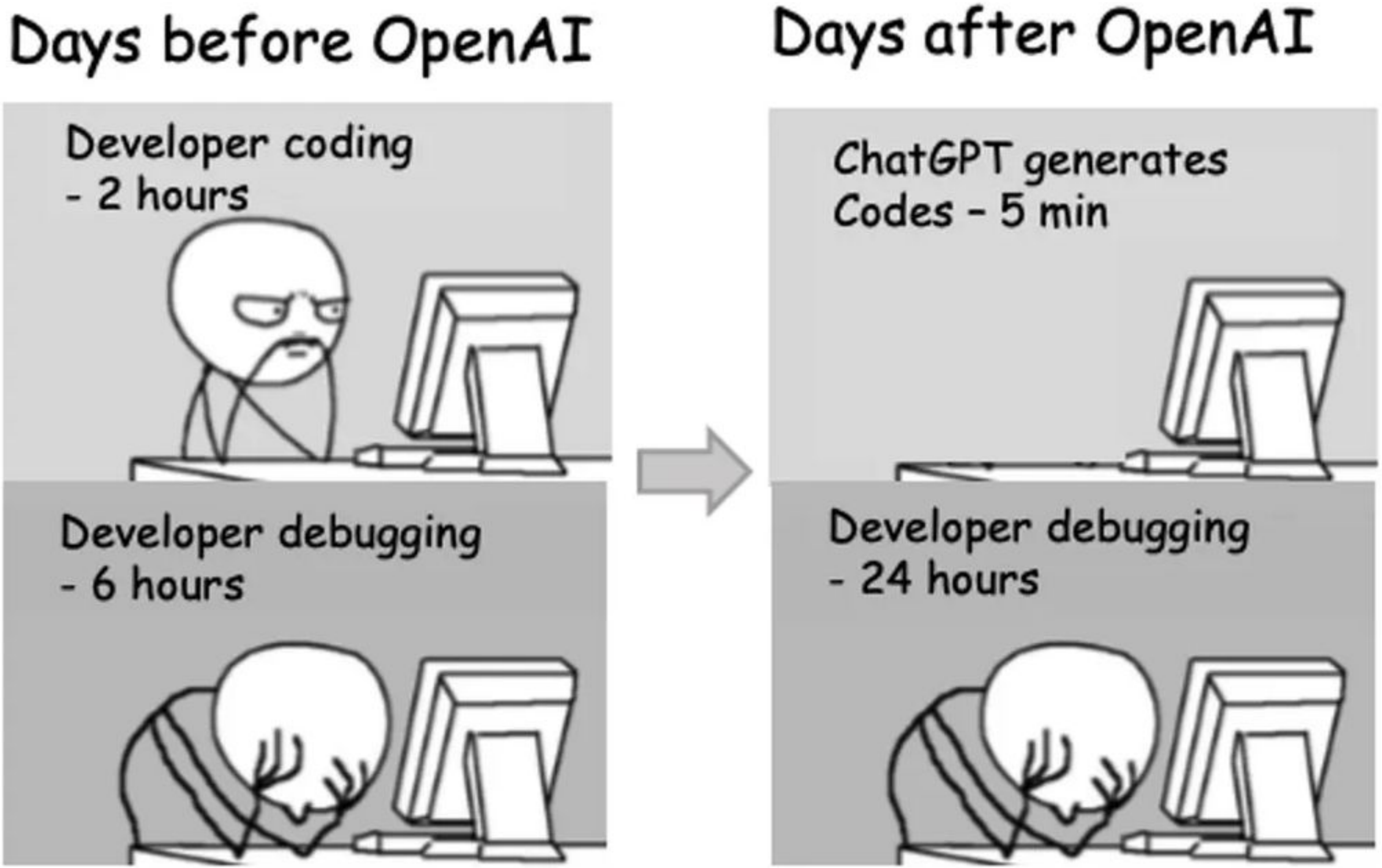
## Transformers are big models

Apart from a few outliers (like DistilBERT), the general strategy to achieve better performance is by increasing the models' sizes as well as the amount of data they are pretrained on.



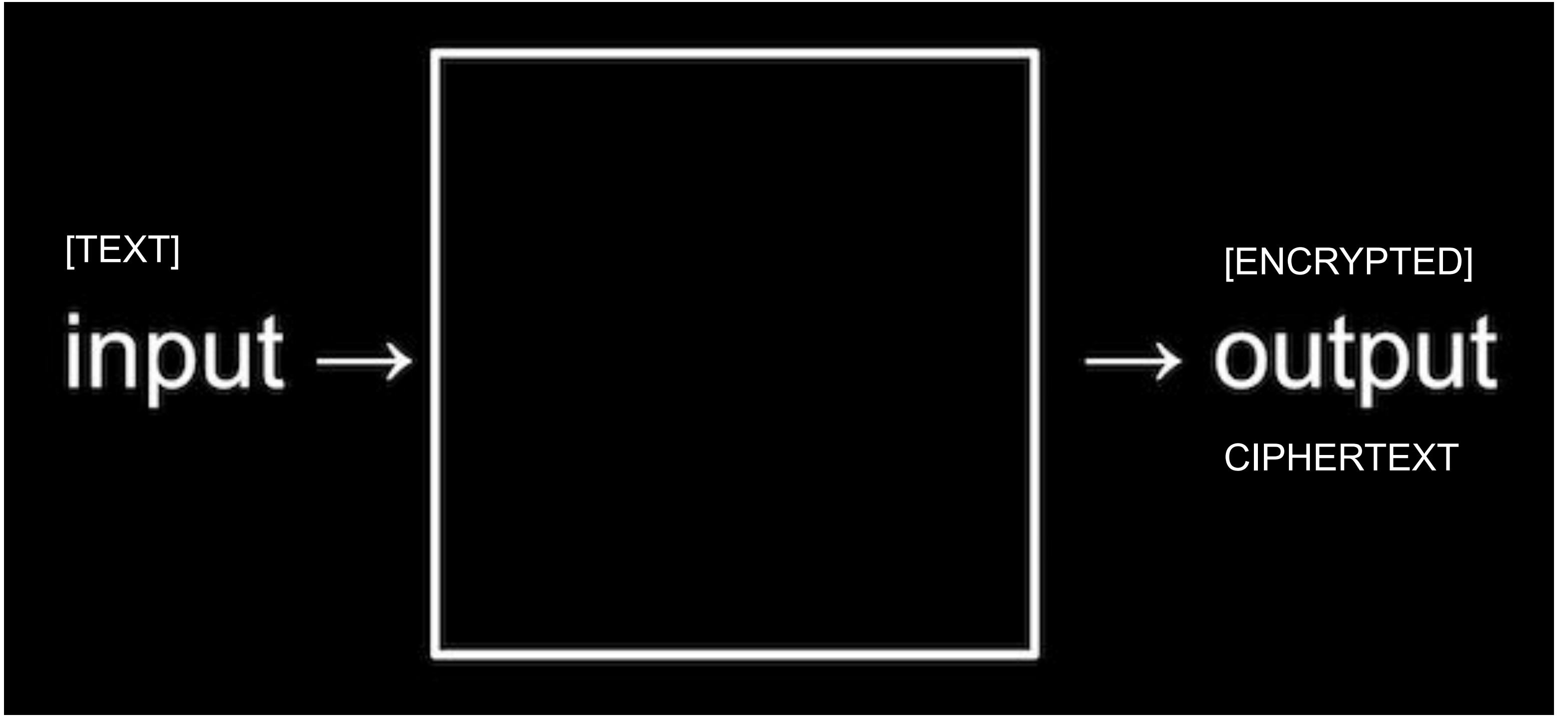

source: huggingface.co

source: AP News
(https://apnews.com/article/power-electricity-amazon-microsoft-power-plants-data-centers-grid-f4763f73bc112425e18f30618dff0039)


Days before OpenAI
Days after OpenAI
Developer coding - 2 hours
ChatGPT generates Codes - 5 min
Developer debugging - 6 hours
Developer debugging - 24 hours


mail.at(0)
mail.at(1)
mail.at(2)
mail.at(3)
mail.at(4)
mail.at(5)
203
204
301
302
303
304

cryptography = hidden writing (Greek)

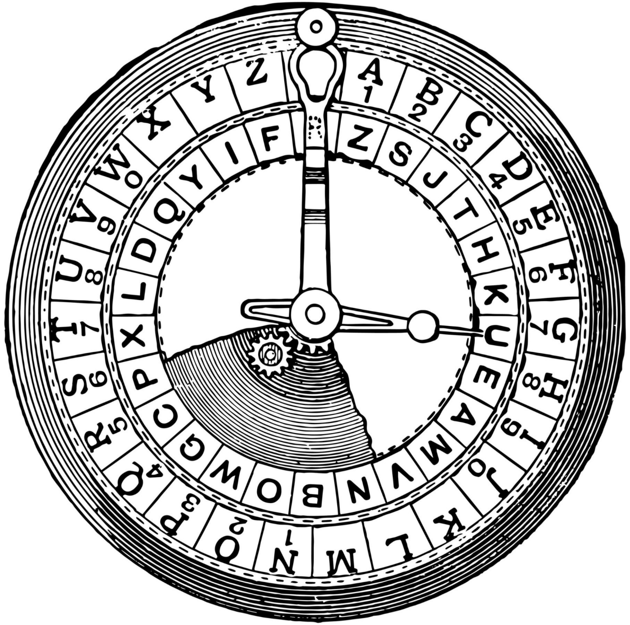



*From Harvard CS50*

# cipher, cypher

- Origins: meant “zero” or circle
- Then meant number, doing arithmetic
- Then meant ***encoding*** text, hidden

# cipher, cypher



# Caesar cipher

Caesar shift

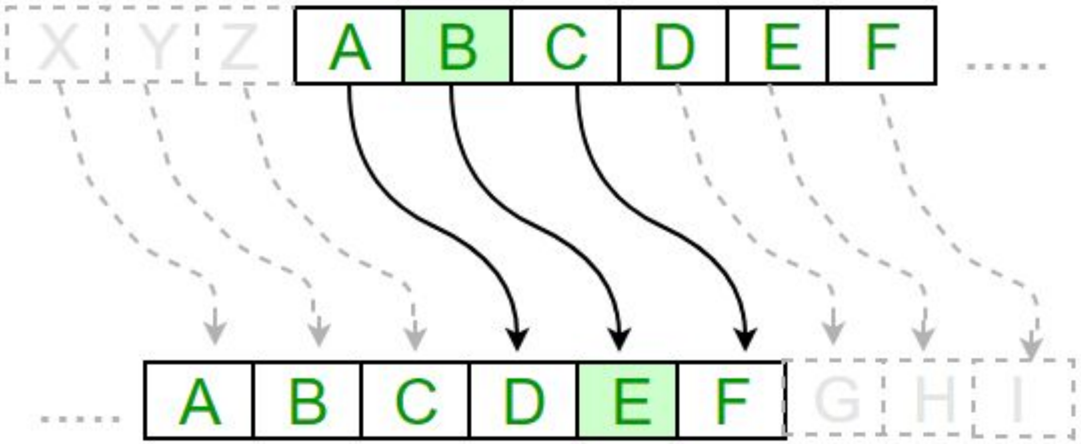
Substitution cipher

military messages, 1st cent. CE

Original Message: “AT DAWN”

SHIFT, or KEY: 3

Encrypted Message: “DW GDZQ”




A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
B A Z Y X W V U T S R Q P O N M L K J I H G F E D C

*Wikimedia*

## Caesar shift in c++ :

- 🔑 shift as input, integer
- Decrypted ciphertext as output
- 2 vectors: 1 container for input text, 1 container for output text
- Nested loops: loop for the word, loop for the character; SHIFT EACH LETTER BY KEY, <INT>
- Alphabet table: ASCII!!

ENCRYPT

🔑 = 1

90 - 65 = 25

25 + 🔑 = 26

26 % 26 = 0

0 + 65 = 65 = A

| | |
|---|---|
| 65 | A |
| 66 | B |
| 67 | C |
| 68 | D |
| 69 | E |
| 70 | F |
| 71 | G |
| 72 | H |
| 73 | I |
| 74 | J |
| 75 | K |
| 76 | L |
| 77 | M |
| 78 | N |
| 79 | O |
| 80 | P |
| 81 | Q |
| 82 | R |
| 83 | S |
| 84 | T |
| 85 | U |
| 86 | V |
| 87 | W |
| 88 | X |
| 89 | Y |
| 90 | Z |
| 91 | [ |

26 letters in alphabet

$$E_n(x) = (x + n) \mod 26.$$

DECRYPT

🔑 = 1

65 - 65 = 0

0 - 🔑 = -1

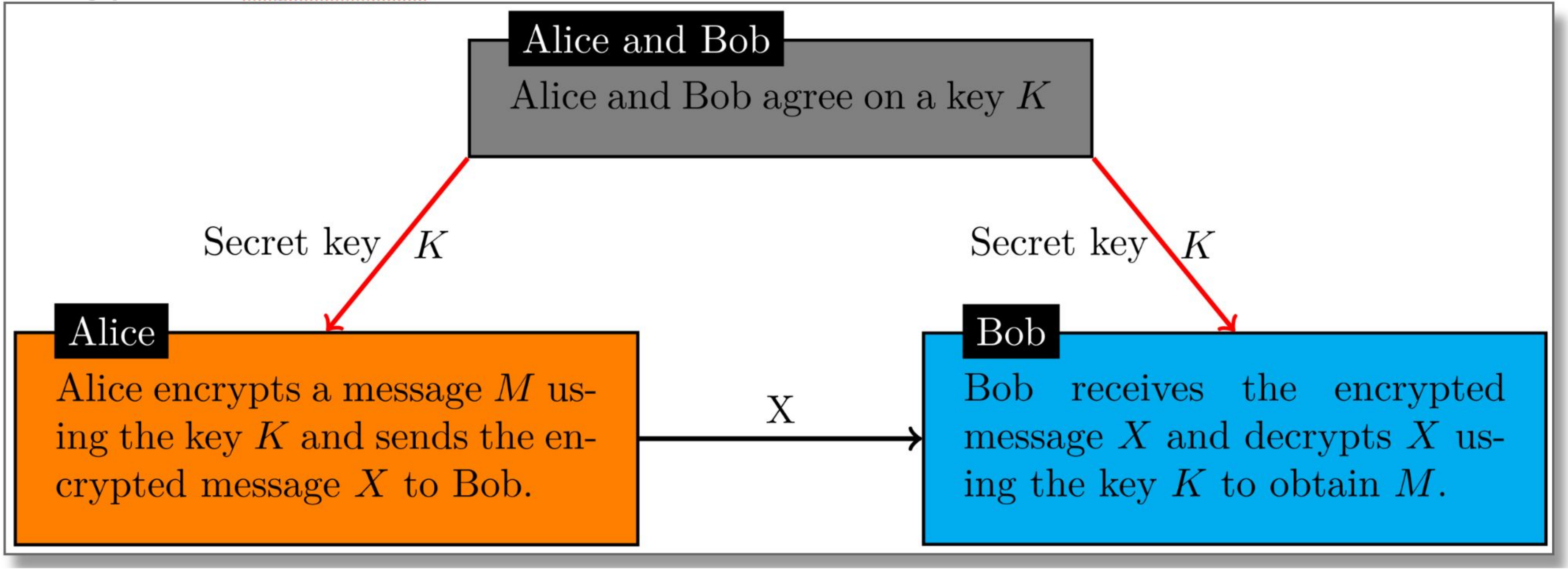
-1 + 26 = 25

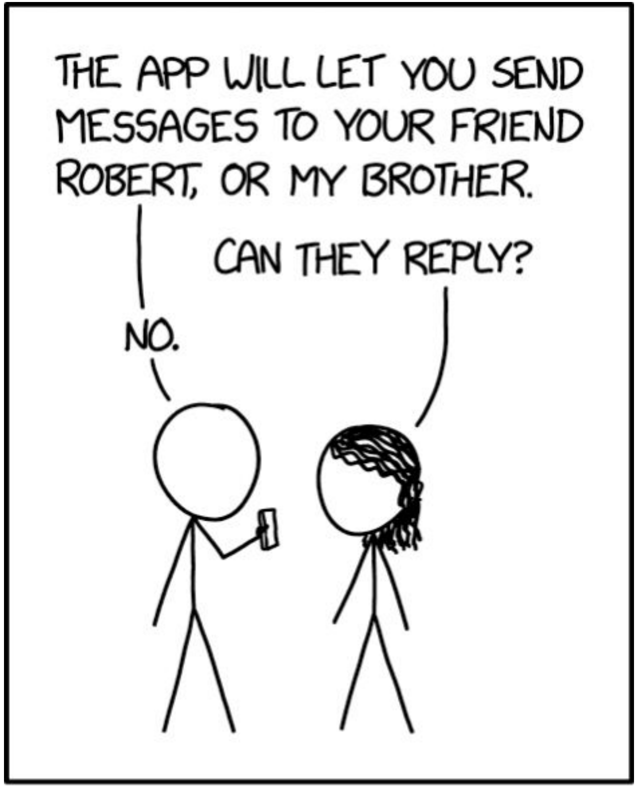
25 % 26 = 25

25 + 65 = 90 = Z

| | |
|---|---|
| 65 | A |
| 66 | B |
| 67 | C |
| 68 | D |
| 69 | E |
| 70 | F |
| 71 | G |
| 72 | H |
| 73 | I |
| 74 | J |
| 75 | K |
| 76 | L |
| 77 | M |
| 78 | N |
| 79 | O |
| 80 | P |
| 81 | Q |
| 82 | R |
| 83 | S |
| 84 | T |
| 85 | U |
| 86 | V |
| 87 | W |
| 88 | X |
| 89 | Y |
| 90 | Z |
| 91 | [ |

26 letters in alphabet


Alice and Bob
Alice and Bob agree on a key $K$
Secret key $K$
Secret key $K$
Alice
Alice encrypts a message $M$ using the key $K$ and sends the encrypted message $X$ to Bob.
X
Bob
Bob receives the encrypted message $X$ and decrypts $X$ using the key $K$ to obtain $M$.


MY NEW SECURE TEXTING APP ONLY ALLOWS PEOPLE NAMED ALICE TO SEND MESSAGES TO PEOPLE NAMED BOB.

# vectors vs. arrays?

```
vector<int> myNumbers(20);

int myNumbers[20];
```

# vectors vs. arrays

- **BIG DIFFERENCE:**
  **YOU CANNOT SIMPLY RESIZE ARRAYS!**

- This makes arrays faster, if you are *really* in need of speedy performance (large, large datasets)

- Arrays don’t need a header #include

- For our purposes, simpler to use dynamic vectors

## myContainer.at(i) vs. myContainer[i]

- .at() function checks the size of your container
- [] does not check the range!
- Both work for vectors
- Only [] works for built-in arrays … BUT …

# **C-strings** vs. **strings** (C++)

```
char myWord[6] = "hello";
string myWord = "hello";
```

- C-string = older, from C, built-in
- Literally array of characters, with '\0' to **END** (size = +1)
- Different library of functions than C++ strings
- Easy to make mistakes with!

## **Why learn arrays + C-strings?**

- We still see them in code (“legacy”)
- Good to understand the most basic data types (like … binary) to know where our more advanced features come from!
- Vectors came from limitations with arrays, the standard template string from limitations with C-strings, etc.!

## 🤖 MIDTERM!

- **Review on 03.23, based on survey**
- **HACKATHON TEAMS also on 03.23, *attendance required***
- **In-class work 03.23, 03.25, Fri. 03.27**
- **Code due 03.27**